

MyBatis Plus 数据库事务代码规范

1. 规范目标

本文规范基于 Spring Boot + MyBatis Plus 的数据库事务使用方式。

核心原则：

1. 原则上推荐使用手动事务。
2. 不推荐在业务代码中直接使用 `@Transactional`。
3. 所有涉及多表写入、多步骤写入、先查后改、状态流转、批量处理、消息消费、定时任务、接口幂等的数据库操作，必须显式声明事务边界。
4. 事务边界必须清晰、可读、可审计、可测试。
5. 事务内只放数据库一致性必须依赖的代码，禁止把远程调用、文件 IO、消息发送、长耗时计算放入事务内。

2. 适用范围

适用于以下场景：

- Spring Boot 项目。
- MyBatis Plus 的 `BaseMapper`、`ServiceImpl`、`IService`、`LambdaQueryWrapper`、`LambdaUpdateWrapper`。
- 多数据源场景中的单数据源本地事务。
- 请求日志、业务流水、审批流、订单、库存、账户、状态机、任务消费、定时补偿等需要保证数据一致性的业务。

不适用于以下场景：

- 分布式事务强一致场景，例如跨多个独立数据库、跨服务同时提交。
- 需要 XA、Seata、TCC、Saga 等专门分布式事务方案的场景。
- 纯查询接口，除非需要强一致读或锁定读。

3. 总体要求

3.1 强制要求

1. 不推荐在 `controller`、`service`、`manager`、`job`、`listener`、`consumer`、`handler` 等业务类方法上使用 `@Transactional`。
2. 不推荐依赖类级别 `@Transactional`。
3. 不推荐在接口方法上声明 `@Transactional`。
4. 禁止通过自调用方式期待事务生效。
5. 禁止在事务内吞异常后继续提交。
6. 禁止在事务内执行远程 HTTP、RPC、MQ 发送、文件上传下载、大对象转换、复杂报表计算。
7. 禁止在事务内使用不受 Spring 管理的数据源连接执行写操作。
8. 禁止事务方法返回前启动未受控异步任务访问同一事务数据。
9. 禁止在事务中混用多个事务管理器，除非代码显式指定且经过评审。
10. 事务失败必须回滚，并且必须保留可诊断日志。

3.2 推荐要求

1. 使用 `TransactionTemplate` 作为手动事务标准实现。
2. 对复杂事务使用专门的 `TxExecutor` 或 `TransactionHelper` 封装统一入口。
3. 事务隔离级别、超时时间、只读属性应按业务显式设置。
4. 写操作必须有明确的幂等键、唯一约束或状态条件。
5. 更新状态必须带原状态条件，避免并发覆盖。
6. 批量写入必须控制批次大小，避免长事务。
7. 事务内异常必须抛出，不允许只记录日志。
8. 事务外发送消息时，推荐使用本地消息表、事务提交后回调或补偿任务。

4. 为什么不推荐使用 `@Transactional`

4.1 事务根本不生效

这是初中级开发者最常遇到的情况：代码写了注解，但数据却并没有像预期那样被事务保护。

同类内部方法调用 (Self-Invocation)： 这是最大的重灾区。如果同一个类中的方法 A 调用了带 `@Transactional` 的方法 B，方法 B 的事务不会生效。因为 Spring 的事务是基于代理对象的，内部调用直接走的是本地 `this` 引用，彻底绕过了外部的 AOP 代理。

作用于非 public 方法： 无论是 JDK 动态代理还是 CGLIB，Spring 默认只拦截并增强 `public` 方法。如果你将 `@Transactional` 加在 `private`、`protected` 或默认可见性的方法上，编译器不会报错，但事务会形同虚设。

方法被 final 或 static 修饰： CGLIB 是通过生成子类来创建代理的，如果方法是 `final` 的，子类无法重写该方法；`static` 方法属于类级别的，同样无法被代理拦截。

错误示例：

```
@Service
public class OrderService {

    public void createOrder(OrderCreateCommand command) {
        this.saveOrder(command);
    }

    @Transactional
    public void saveOrder(OrderCreateCommand command) {
        // 这里通过 this.saveOrder 调用时，事务不会生效
    }
}
```

4.2 事务不回滚

系统确实开启了事务，甚至也抛出了异常，但数据库的修改依然被提交了。

异常类型不匹配： 默认情况下，Spring 事务只在遇到 `RuntimeException` 和 `Error` 时才会触发回滚。如果代码抛出了普通的 `Exception`，例如 `IOException`、`SQLException`，事务默认不会回滚。

纠正方案：养成显式声明的习惯，写成：

```
@Transactional(rollbackFor = Exception.class)
```

异常被吞没 (Swallowed Exception)： 如果在事务方法内部使用 `try-catch` 捕获了异常，并且没有把异常重新 `throw` 抛出去，Spring AOP 代理层在外部看到的是方法正常执行完毕，就会理所当然地提交事务。

错误示例：

```
@Transactional(rollbackFor = Exception.class)
public void updateData() {
    try {
        mapper.updateById(entity);
    } catch (Exception e) {
        log.error("更新失败", e);
        // 异常被吞掉，事务方法正常返回，事务会提交
    }
}
```

4.3 性能危机与系统崩溃

这类问题在测试环境往往难以发现，一旦上了生产环境遇到高并发，可能直接导致系统宕机。

大事务 (Long Transactions)：如果在 `@Transactional` 标注的方法中，不仅进行了数据库读写，还包含耗时的外部 RPC 调用、HTTP 请求、复杂的文件 I/O 或漫长的休眠逻辑，会导致数据库连接被长时间占用。并发量一上来，数据库连接池可能瞬间耗尽，其他所有需要访问数据库的业务全部卡死。

纠正方案：尽早提交事务。将核心数据库操作剥离出来单独加事务，耗时的非数据库操作放在事务之外。

死锁风险升高：大事务持有行锁或表锁的时间过长，如果有其他并发操作更新相同的数据，极其容易触发数据库级别的死锁。

4.4 并发与架构复杂性问题

多线程跨线程失效：Spring 事务的核心机制依赖 `ThreadLocal` 绑定数据库连接。如果在一个事务方法中开启新的子线程，例如 `CompletableFuture` 或 `new Thread()`，子线程获取不到父线程的事务上下文。它们将处于完全不同的数据库连接中，无法做到统一回滚。

嵌套事务传播行为混乱：当多个事务方法相互调用时，如果没有理清 `Propagation.REQUIRED`、`REQUIRES_NEW`、`NESTED` 等传播行为的含义，很容易出现“某个子方法报错，导致整个大循环的成功数据也被无辜连累回滚”的逻辑 Bug。

5. 如仍使用 `@Transactional` 的注意事项

原则上新代码不推荐使用 `@Transactional`。历史代码短期无法改造，或确有必要使用时，必须满足以下要求。

5.1 必须显式声明回滚异常

```
@Transactional(rollbackFor = Exception.class)
public void updateOrder() {
    // 历史代码临时保留
}
```

不推荐只写：

```
@Transactional
public void updateOrder() {
}
```

5.2 必须用在 public 方法上

`@Transactional` 只能放在被 Spring 代理正常拦截的 `public` 方法上。

不应写成：

```
@Transactional
private void saveData() {
}
```

5.3 不应同类内部调用事务方法

错误示例：

```
public void submit() {
    this.doSubmit();
}

@Transactional(rollbackFor = Exception.class)
public void doSubmit() {
}
```

5.4 多数据源必须指定事务管理器

```
@Transactional(
    transactionManager = "orderTransactionManager",
    rollbackFor = Exception.class
)
public void createOrder() {
}
```

5.5 必须控制事务范围

事务方法内禁止包含：

- HTTP 调用。
- RPC 调用。
- MQ 发送。
- 文件读写。
- 大批量 Excel 解析。
- `Thread.sleep`。
- 等待锁、等待队列、等待外部系统。

5.6 异常不能被吞掉

错误示例：

```
@Transactional(rollbackFor = Exception.class)
public void updateData() {
    try {
        mapper.updateById(entity);
    } catch (Exception e) {
        log.error("update failed", e);
        // 吞掉异常会导致事务提交
    }
}
```

正确示例：

```
@Transactional(rollbackFor = Exception.class)
public void updateData() {
    try {
        mapper.updateById(entity);
    } catch (Exception e) {
        log.error("update failed", e);
        throw e;
    }
}
```

6. 手动事务标准方案

6.1 推荐使用 `TransactionTemplate`

`TransactionTemplate` 是 Spring 官方提供的程式化事务工具。它比手动获取连接更安全，能复用 Spring 的事务管理、连接绑定、异常处理、资源释放能力。

标准依赖：

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.support.TransactionTemplate;
```

标准注入：

```
@Service
public class OrderApplicationService {

    private final TransactionTemplate transactionTemplate;

    public OrderApplicationService(TransactionTemplate transactionTemplate) {
        this.transactionTemplate = transactionTemplate;
    }
}
```

6.2 标准事务配置

推荐在配置类中统一配置默认事务模板。

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```

import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.support.TransactionTemplate;

@Configuration
public class TransactionConfig {

    @Bean
    public TransactionTemplate transactionTemplate(PlatformTransactionManager
transactionManager) {
        TransactionTemplate template = new
TransactionTemplate(transactionManager);

        template.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);

        template.setIsolationLevel(TransactionDefinition.ISOLATION_READ_COMMITTED);
        template.setTimeout(10);
        return template;
    }
}

```

配置说明：

配置项	推荐值	说明
propagationBehavior	PROPAGATION_REQUIRED	默认加入当前事务，没有则创建新事务
isolationLevel	ISOLATION_READ_COMMITTED	常规业务推荐，避免脏读
timeout	5 到 30 秒	按业务设置，禁止无限等待
readOnly	写事务为 false	查询事务可单独设置为 true

7. 标准 Demo：创建订单

7.1 业务要求

创建订单时需要同时完成：

1. 校验用户和商品。
2. 扣减库存。
3. 创建订单主表。
4. 创建订单明细表。
5. 写入订单状态流水。

其中第 2 到第 5 步必须在同一个数据库事务中完成。第 1 步中不涉及锁和写入的校验应放在事务外。

7.2 Mapper 示例

```

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import org.apache.ibatis.annotations.Mapper;

@Mapper
public interface OrderMapper extends BaseMapper<OrderDO> {
}

```

```

@Mapper
public interface OrderItemMapper extends BaseMapper<OrderItemDO> {
}

@Mapper
public interface ProductMapper extends BaseMapper<ProductDO> {
}

@Mapper
public interface OrderStatusLogMapper extends BaseMapper<OrderStatusLogDO> {
}

```

7.3 手动事务标准写法

```

import com.baomidou.mybatisplus.core.conditions.update.LambdaUpdateWrapper;
import org.springframework.stereotype.Service;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.support.TransactionTemplate;

import java.time.LocalDateTime;
import java.util.List;

@Service
public class OrderApplicationService {

    private final TransactionTemplate transactionTemplate;
    private final ProductMapper productMapper;
    private final OrderMapper orderMapper;
    private final OrderItemMapper orderItemMapper;
    private final OrderStatusLogMapper orderStatusLogMapper;

    public OrderApplicationService(
        TransactionTemplate transactionTemplate,
        ProductMapper productMapper,
        OrderMapper orderMapper,
        OrderItemMapper orderItemMapper,
        OrderStatusLogMapper orderStatusLogMapper) {
        this.transactionTemplate = transactionTemplate;
        this.productMapper = productMapper;
        this.orderMapper = orderMapper;
        this.orderItemMapper = orderItemMapper;
        this.orderStatusLogMapper = orderStatusLogMapper;
    }

    public Long createOrder(CreateOrderCommand command) {
        validateCommand(command);

        ProductDO product = productMapper.selectById(command.getProductId());
        if (product == null) {
            throw new BizException("商品不存在");
        }
        if (!Boolean.TRUE.equals(product.getEnabled())) {
            throw new BizException("商品已下架");
        }
    }
}

```

```

        TransactionTemplate writeTx = new
TransactionTemplate(transactionTemplate.getTransactionManager());

writeTx.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);

writeTx.setIsolationLevel(TransactionDefinition.ISOLATION_READ_COMMITTED);
writeTx.setTimeout(10);

return writeTx.execute(status -> {
    try {
        int stockUpdated = productMapper.update(
            null,
            new LambdaUpdateWrapper<ProductDO>()
                .eq(ProductDO::getId, command.getProductId())
                .ge(ProductDO::getStock, command.getQuantity())
                .setSql("stock = stock - " + command.getQuantity())
        );
        if (stockUpdated != 1) {
            throw new BizException("库存不足");
        }

        OrderDO order = new OrderDO();
        order.setUserId(command.getUserId());
        order.setProductId(command.getProductId());
        order.setQuantity(command.getQuantity());
        order.setStatus(OrderStatus.CREATED.name());
        order.setCreateTime(LocalDateTime.now());
        order.setUpdateTime(LocalDateTime.now());
        orderMapper.insert(order);

        OrderItemDO item = new OrderItemDO();
        item.setOrderId(order.getId());
        item.setProductId(command.getProductId());
        item.setQuantity(command.getQuantity());
        item.setCreateTime(LocalDateTime.now());
        orderItemMapper.insert(item);

        OrderStatusLogDO statusLog = new OrderStatusLogDO();
        statusLog.setOrderId(order.getId());
        statusLog.setFromStatus(null);
        statusLog.setToStatus(OrderStatus.CREATED.name());
        statusLog.setOperateTime(LocalDateTime.now());
        statusLog.setRemark("创建订单");
        orderStatusLogMapper.insert(statusLog);

        return order.getId();
    } catch (RuntimeException e) {
        status.setRollbackOnly();
        throw e;
    } catch (Exception e) {
        status.setRollbackOnly();
        throw new BizException("创建订单失败", e);
    }
});
}

private void validateCommand(CreateOrderCommand command) {
    if (command == null) {

```



```

        throw new BizException("请求参数不能为空");
    }
    if (command.getUserId() == null) {
        throw new BizException("用户ID不能为空");
    }
    if (command.getProductId() == null) {
        throw new BizException("商品ID不能为空");
    }
    if (command.getQuantity() == null || command.getQuantity() <= 0) {
        throw new BizException("购买数量必须大于0");
    }
}
}
}

```

7.4 Demo 关键点

1. 参数校验放在事务外。
2. 普通商品状态查询放在事务外。
3. 扣减库存、创建订单、创建明细、写流水放在同一个事务内。
4. 库存扣减使用条件更新，不使用先查库存再更新库存。
5. 事务内捕获异常后必须 `setRollbackOnly()` 并继续抛出。
6. 对受检异常统一包装成运行时业务异常，保证外层能感知失败。
7. 事务超时时间显式设置。

8. 推荐封装：统一事务执行器

为了避免每个业务方法重复创建 `TransactionTemplate`，项目统一推荐使用 `UnifiedTransactionExecutor` 执行业务事务。业务代码原则上不直接使用 `@Transactional`，也不直接调用 `PlatformTransactionManager#getTransaction`、`commit`、`rollback`。

`UnifiedTransactionExecutor` 的定位：

1. 对业务层提供统一的手动事务入口。
2. 统一设置事务传播行为、隔离级别、超时时间。
3. 统一处理受检异常、运行时异常和 `Error` 的回滚。
4. 统一记录慢事务日志，辅助发现长事务和潜在锁等待。
5. 提供 `afterCommit` 能力，把缓存删除、普通通知等短任务放到事务提交后执行。
6. `afterCommit` 只传播 `MDC` 日志上下文，不传播 `RequestContextHolder`、`HttpServletRequest` 或 Spring Security 上下文。
7. 关键 MQ、必须送达的业务事件、跨系统强依赖动作必须使用本地消息表或可靠任务表，不应只依赖内存回调。

8.1 UnifiedTransactionExecutor 使用边界

推荐使用：

1. 多表写入需要原子提交。
2. 状态流转和状态流水需要一致。
3. 库存、余额、额度等资源扣减。
4. 幂等记录和业务处理需要同事务提交。
5. 本地消息表和业务数据需要同事务提交。
6. 定时任务、消息消费、接口处理中的短事务写入。
7. 需要事务提交后执行短任务，例如删除缓存、记录轻量日志、触发非关键内存通知。

谨慎使用：

1. `requiresNew` 会挂起外层事务，可能导致内外事务提交结果不一致。
2. `readOnly` 只适合短查询事务，不适合报表和导出。
3. 自定义超时不能掩盖 SQL 慢、索引缺失或锁等待问题。
4. 多数据源场景需要确认注入的 `PlatformTransactionManager` 是否对应正确数据源。
5. `afterCommit` 适合短任务，不适合慢任务、阻塞任务、必须送达任务。
6. `afterCommit` 不传播请求上下文，不能在回调中通过 `RequestContextHolder` 读取请求参数、Header、用户信息。

禁止使用方式：

1. 禁止在事务内调用远程 HTTP、RPC。
2. 禁止在事务内直接发送 MQ。
3. 禁止在事务内删除 Redis 缓存。
4. 禁止在事务内执行文件 IO。
5. 禁止在事务内执行大批量循环且不拆批。
6. 禁止在 lambda 内捕获异常后不抛出。
7. 禁止用返回 `false` 替代抛异常来表达失败。
8. 禁止在 `afterCommit` 中处理出账、支付、库存同步等关键一致性动作。
9. 禁止在 `afterCommit` 中读取 `HttpServletRequest`、`RequestContextHolder`。

8.2 UnifiedTransactionExecutor 标准实现

项目中的统一事务执行器位于：

```
com.yunjin.centerframework.db.until.UnifiedTransactionExecutor
```

标准实现要点：

1. 基于 `TransactionTemplate` 托管提交和回滚。
2. 基于实例级 `afterCommitExecutor` 执行提交后短任务，避免静态线程池在 Spring 多上下文测试中被提前关闭。
3. `afterCommitExecutor` 使用 `CallerRunsPolicy`，队列满时由当前线程执行回调，避免阻塞等待队列空间造成连接池与线程池环路死锁。
4. 异步回调只恢复 `MDC`，不跨线程传递 Servlet 请求对象。
5. 回调异常只记录日志，不向业务调用方传播，避免“数据库已提交但接口返回失败”。

核心实现摘要：

```
package com.yunjin.centerframework.db.until;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.stereotype.Component;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.support.TransactionSynchronization;
import org.springframework.transaction.support.TransactionSynchronizationManager;
import org.springframework.transaction.support.TransactionTemplate;

import java.util.Map;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.LinkedBlockingQueue;
```

```

import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ThreadFactory;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * 统一编程式事务执行器。
 *
 * <p>业务代码应优先使用该执行器管理事务，避免直接使用 {@code @Transactional}
 * 或直接调用 {@link PlatformTransactionManager} 的 commit/rollback。</p>
 *
 * <p>该类基于 {@link TransactionTemplate} 执行事务，统一处理提交、回滚、超时和慢事务日
 * 志。
 * 业务方只需要在 lambda 中编写必须原子提交的数据库操作。</p>
 *
 * @author taozhangxian
 */
@Component
public class UnifiedTransactionExecutor implements DisposableBean {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(UnifiedTransactionExecutor.class);

    private static final int DEFAULT_TIMEOUT_SECONDS = 10;
    private static final int READ_ONLY_TIMEOUT_SECONDS = 5;
    private static final long SLOW_TRANSACTION_WARN_MILLIS = 1000L;

    private final PlatformTransactionManager transactionManager;
    private final ExecutorService afterCommitExecutor;

    public UnifiedTransactionExecutor(PlatformTransactionManager
transactionManager) {
        this.transactionManager = transactionManager;
        this.afterCommitExecutor = new ThreadPoolExecutor(
            2,
            4,
            60L,
            TimeUnit.SECONDS,
            new LinkedBlockingQueue<>(1000),
            new NamedThreadFactory(),
            new ThreadPoolExecutor.CallerRunsPolicy());
    }

    public <T> T required(String transactionName, TxSupplier<T> supplier) {
        return execute(transactionName,
            TransactionDefinition.PROPGATION_REQUIRED,
            TransactionDefinition.ISOLATION_READ_COMMITTED,
            DEFAULT_TIMEOUT_SECONDS, false, supplier);
    }

    public void required(String transactionName, TxRunnable runnable) {
        required(transactionName, () -> {
            runnable.run();
            return null;
        });
    }
}

```

```

    public <T> T required(String transactionName, int timeoutSeconds,
TxSupplier<T> supplier) {
        return execute(transactionName,
TransactionDefinition.PROPGATION_REQUIRED,
TransactionDefinition.ISOLATION_READ_COMMITTED, timeoutSeconds,
false, supplier);
    }

    public void required(String transactionName, int timeoutSeconds, TxRunnable
runnable) {
        required(transactionName, timeoutSeconds, () -> {
            runnable.run();
            return null;
        });
    }

    public <T> T requiresNew(String transactionName, TxSupplier<T> supplier) {
        if (TransactionSynchronizationManager.isActualTransactionActive()) {
            LOGGER.warn("Nested requiresNew transaction detected, name={}. "
+ "It will request another database connection and may
exhaust the connection pool "
+ "under high concurrency.", transactionName);
        }
        return execute(transactionName,
TransactionDefinition.PROPGATION_REQUIRES_NEW,
TransactionDefinition.ISOLATION_READ_COMMITTED,
DEFAULT_TIMEOUT_SECONDS, false, supplier);
    }

    public void requiresNew(String transactionName, TxRunnable runnable) {
        requiresNew(transactionName, () -> {
            runnable.run();
            return null;
        });
    }

    public <T> T readOnly(String transactionName, TxSupplier<T> supplier) {
        if (TransactionSynchronizationManager.isActualTransactionActive()) {
            LOGGER.warn("Nested readOnly transaction detected, name={}. "
+ "readOnly=true may be ignored when joining an existing
read-write transaction.",
transactionName);
        }
        return execute(transactionName,
TransactionDefinition.PROPGATION_REQUIRED,
TransactionDefinition.ISOLATION_READ_COMMITTED,
READ_ONLY_TIMEOUT_SECONDS, true, supplier);
    }

    /**
     * 注册事务提交后的短任务。
     *
     * <p>不要在回调中依赖 RequestContextHolder 或 HttpServletRequest。
     * Servlet 请求对象在响应完成后可能被容器回收复用。
     * 需要租户、用户、traceId 等数据时，应在注册回调前提取为普通变量并传入 lambda。</p>
     */
    public void afterCommit(TxRunnable runnable) {
        if (!TransactionSynchronizationManager.isSynchronizationActive())

```

```

        ||
!TransactionSynchronizationManager.isActualTransactionActive() {
    submitAfterCommitAction("no-active-transaction", runnable);
    return;
}
TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
    @Override
    public void afterCommit() {
        submitAfterCommitAction("after-commit", runnable);
    }
});
}

public boolean isActualTransactionActive() {
    return TransactionSynchronizationManager.isActualTransactionActive();
}

private <T> T execute(String transactionName, int propagationBehavior, int
isolationLevel,
    int timeoutSeconds, boolean readOnly, TxSupplier<T> supplier) {
    if (supplier == null) {
        throw new IllegalArgumentException("transaction supplier must not be
null");
    }
    int actualTimeoutSeconds = timeoutSeconds > 0 ? timeoutSeconds :
DEFAULT_TIMEOUT_SECONDS;
    TransactionTemplate template = new
TransactionTemplate(transactionManager);
    template.setName(transactionName);
    template.setPropagationBehavior(propagationBehavior);
    template.setIsolationLevel(isolationLevel);
    template.setTimeout(actualTimeoutSeconds);
    template.setReadOnly(readOnly);

    long startMillis = System.currentTimeMillis();
    try {
        return template.execute(status -> {
            try {
                return supplier.get();
            } catch (RuntimeException ex) {
                status.setRollbackOnly();
                throw ex;
            } catch (Error error) {
                status.setRollbackOnly();
                throw error;
            } catch (Exception ex) {
                status.setRollbackOnly();
                throw new TransactionExecutionException("Transaction
execution failed: " + transactionName, ex);
            }
        });
    } finally {
        long costMillis = System.currentTimeMillis() - startMillis;
        if (costMillis > SLOW_TRANSACTION_WARN_MILLIS) {
            LOGGER.warn("slow transaction detected, name={}, costMillis={},
timeoutSeconds={}, readOnly=",

```

```

        transactionName, costMillis, actualTimeoutSeconds,
readOnly);
    }
}

private void submitAfterCommitAction(String source, TxRunnable runnable) {
    if (runnable == null) {
        LOGGER.warn("Ignore null afterCommit action, source={}", source);
        return;
    }
    try {

afterCommitExecutor.execute(createContextAwareAfterCommitTask(source,
runnable));
        } catch (RejectedExecutionException ex) {
            LOGGER.error("After commit action rejected, source={}, executor may
be shutdown", source, ex);
        }
    }

    private Runnable createContextAwareAfterCommitTask(String source, TxRunnable
runnable) {
        Map<String, String> capturedMdcContext = MDC.getCopyOfContextMap();

        return () -> {
            Map<String, String> oldMdcContext = MDC.getCopyOfContextMap();
            try {
                restoreMdc(capturedMdcContext);
                try {
                    runnable.run();
                } catch (Throwable ex) {
                    LOGGER.error("After commit action failed, source={}",
source, ex);
                }
            } finally {
                restoreMdc(oldMdcContext);
            }
        };
    }

    private void restoreMdc(Map<String, String> mdcContext) {
        if (mdcContext == null) {
            MDC.clear();
        } else {
            MDC.setContextMap(mdcContext);
        }
    }

    @FunctionalInterface
    public interface TxSupplier<T> {
        T get() throws Exception;
    }

    @FunctionalInterface
    public interface TxRunnable {
        void run() throws Exception;
    }

```

```

public static class TransactionExecutionException extends RuntimeException {

    public TransactionExecutionException(String message, Throwable cause) {
        super(message, cause);
    }
}

@Override
public void destroy() {
    afterCommitExecutor.shutdown();
}

private static class NamedThreadFactory implements ThreadFactory {

    private final AtomicInteger counter = new AtomicInteger(1);

    @Override
    public Thread newThread(Runnable runnable) {
        Thread thread = new Thread(runnable,
            "unified-transaction-after-commit-" +
counter.getAndIncrement());
        thread.setDaemon(true);
        return thread;
    }
}
}

```

8.3 UnifiedTransactionExecutor 方法说明

方法	场景	事务特性
<code>required(String, TxSupplier<T>)</code>	有返回值的普通写事务	<code>PROPAGATION_REQUIRED</code> 、 <code>READ_COMMITTED</code> 、默认 10 秒
<code>required(String, TxRunnable)</code>	无返回值的普通写事务	<code>PROPAGATION_REQUIRED</code> 、 <code>READ_COMMITTED</code> 、默认 10 秒
<code>required(String, int, TxSupplier<T>)</code>	需要自定义超时的写事务	超时时间由调用方指定
<code>required(String, int, TxRunnable)</code>	无返回值且需要自定义超时的写事务	超时时间由调用方指定
<code>requiresNew(String, TxSupplier<T>)</code>	独立提交的事务	挂起当前事务并开启新事务
<code>requiresNew(String, TxRunnable)</code>	独立提交且无返回值的事务	挂起当前事务并开启新事务
<code>readOnly(String, TxSupplier<T>)</code>	需要事务一致性的只读查询	<code>readOnly=true</code> 、默认 5 秒
<code>afterCommit(TxRunnable)</code>	事务提交后短任务	有事务则提交后异步执行，无事务也异步执行；只传播 <code>MDC</code>

方法	场景	事务特性
<code>isActualTransactionActive()</code>	判断当前线程是否存在真实事务	用于边界检查和诊断

8.4 UnifiedTransactionExecutor 标准 Demo：取消订单

```
import com.baomidou.mybatisplus.core.conditions.update.LambdaUpdateWrapper;
import com.yunjin.centerframework.db.until.UnifiedTransactionExecutor;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;

@Service
public class OrderCancelService {

    private final UnifiedTransactionExecutor transactionExecutor;
    private final OrderMapper orderMapper;
    private final ProductMapper productMapper;
    private final OrderStatusLogMapper orderStatusLogMapper;

    public OrderCancelService(
        UnifiedTransactionExecutor transactionExecutor,
        OrderMapper orderMapper,
        ProductMapper productMapper,
        OrderStatusLogMapper orderStatusLogMapper) {
        this.transactionExecutor = transactionExecutor;
        this.orderMapper = orderMapper;
        this.productMapper = productMapper;
        this.orderStatusLogMapper = orderStatusLogMapper;
    }

    public void cancelOrder(Long orderId, Long operatorId) {
        if (orderId == null) {
            throw new BizException("订单ID不能为空");
        }

        transactionExecutor.required("cancelOrder", () -> {
            OrderDO order = orderMapper.selectById(orderId);
            if (order == null) {
                throw new BizException("订单不存在");
            }
            if (!OrderStatus.CREATED.name().equals(order.getStatus())) {
                throw new BizException("当前订单状态不允许取消");
            }

            int orderUpdated = orderMapper.update(
                null,
                new LambdaUpdateWrapper<OrderDO>()
                    .eq(OrderDO::getId, orderId)
                    .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
            );
        });
    }
}
```

```

        .set(OrderDO::getStatus, OrderStatus.CANCELED.name())
        .set(OrderDO::getUpdateTime, LocalDateTime.now())
    );
    if (orderUpdated != 1) {
        throw new BizException("订单状态已变化，请刷新后重试");
    }

    int stockUpdated = productMapper.update(
        null,
        new LambdaUpdateWrapper<ProductDO>()
            .eq(ProductDO::getId, order.getProductId())
            .setSql("stock = stock + " + order.getQuantity())
    );
    if (stockUpdated != 1) {
        throw new BizException("库存回补失败");
    }

    OrderStatusLogDO log = new OrderStatusLogDO();
    log.setOrderId(orderId);
    log.setFromStatus(OrderStatus.CREATED.name());
    log.setToStatus(OrderStatus.CANCELED.name());
    log.setOperatorId(operatorId);
    log.setOperateTime(LocalDateTime.now());
    log.setRemark("取消订单");
    orderStatusLogMapper.insert(log);
}
}
}

```

关键点：

1. `cancelOrder` 参数校验放在事务外。
2. 事务名称使用明确业务动作，例如 `cancelOrder`。
3. 状态更新必须带原状态条件。
4. 每个关键 `update` 都检查影响行数。
5. lambda 内抛出异常即可回滚，不允许吞异常。
6. 事务内不调用 MQ、Redis、HTTP、RPC、文件 IO。

8.5 UnifiedTransactionExecutor 有返回值 Demo

```

public Long createOrder(CreateOrderCommand command) {
    validateCreateOrderCommand(command);

    Long orderId = transactionExecutor.required("createOrder", () -> {
        int stockUpdated = productMapper.update(
            null,
            new LambdaUpdateWrapper<ProductDO>()
                .eq(ProductDO::getId, command.getProductId())
                .ge(ProductDO::getStock, command.getQuantity())
                .setSql("stock = stock - " + command.getQuantity())
        );
        if (stockUpdated != 1) {
            throw new BizException("库存不足");
        }

        OrderDO order = new OrderDO();
    });
}

```

```

        order.setUserId(command.getUserId());
        order.setProductId(command.getProductId());
        order.setQuantity(command.getQuantity());
        order.setStatus(OrderStatus.CREATED.name());
        order.setCreateTime(LocalDateTime.now());
        orderMapper.insert(order);

        return order.getId();
    });

    return orderId;
}

```

8.6 UnifiedTransactionExecutor 自定义超时 Demo

普通写事务默认 10 秒。批量但仍可控的短事务可以显式指定超时时间。

```

public void importOrderItems(Long orderId, List<OrderItemDO> items) {
    if (items.size() > 500) {
        throw new BizException("单批明细不能超过500条");
    }

    transactionExecutor.required("importOrderItems", 20, () -> {
        for (OrderItemDO item : items) {
            item.setOrderId(orderId);
            orderItemMapper.insert(item);
        }
    });
}

```

注意：

1. 自定义超时不是放开长事务的理由。
2. 大批量导入必须拆批，每批单独事务。
3. 单批数量必须有上限。
4. 超时时间应小于调用方可接受的等待时间。

8.7 UnifiedTransactionExecutor 提交后回调 Demo

缓存删除、普通异步通知可以放在事务提交后执行。关键 MQ、出账消息、必须送达的业务事件，应使用本地消息表或可靠任务表。

注意：afterCommit 不会传播 RequestContextHolder 或 HttpServletRequest。如果回调需要用户、租户、traceId 等数据，必须在注册回调前提取为普通变量。

8.7.1 使用边界

1. afterCommit 中的逻辑不参与数据库事务。
2. afterCommit 执行失败不会回滚已提交的数据库事务。
3. 关键消息推荐使用本地消息表，而不是只依赖内存回调。
4. afterCommit 不适合放置必须强一致的数据写入。
5. afterCommit 只传播 MDC，不传播 RequestContextHolder、HttpServletRequest。
6. 回调必须是短任务；线程池满载时可能由提交线程执行回调。
7. 回调中禁止直接读取当前请求对象；需要的业务上下文必须提前提取。

8.7.2 标准 Demo

```

public void payOrder(Long orderId, Long operatorId) {
    String cacheKey = "order:" + orderId;
    String traceId = TraceContext.getTraceId();

    transactionExecutor.required("payOrder", () -> {
        int updated = orderMapper.update(
            null,
            new LambdaUpdateWrapper<OrderDO>()
                .eq(OrderDO::getId, orderId)
                .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
                .set(OrderDO::getStatus, OrderStatus.PAID.name())
                .set(OrderDO::getUpdateTime, LocalDateTime.now())
        );
        if (updated != 1) {
            throw new BizException("订单状态不允许支付");
        }

        transactionExecutor.afterCommit(() -> {
            redisTemplate.delete(cacheKey);
            log.info("payOrder cache deleted, orderId={}, operatorId={},
traceId={}",
                orderId, operatorId, traceId);
        });
    });
}

```

8.7.3 本地消息表推荐写法

```

public void payOrderWithReliableMessage(Long orderId) {
    transactionExecutor.required("payOrderWithReliableMessage", () -> {
        int updated = orderMapper.update(
            null,
            new LambdaUpdateWrapper<OrderDO>()
                .eq(OrderDO::getId, orderId)
                .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
                .set(OrderDO::getStatus, OrderStatus.PAID.name())
                .set(OrderDO::getUpdateTime, LocalDateTime.now())
        );
        if (updated != 1) {
            throw new BizException("订单状态不允许支付");
        }

        LocalMessageDO message = new LocalMessageDO();
        message.setMessageId(IdGenerator.nextId());
        message.setTopic("order.paid");
        message.setBody(JsonUtils.toJson(buildOrderPaidMessage(orderId)));
        message.setStatus(MessageStatus.PENDING.name());
        message.setCreateTime(LocalDateTime.now());
        LocalMessageMapper.insert(message);
    });
}

```

8.8 UnifiedTransactionExecutor 只读事务 Demo

只读事务适用于需要一个一致性事务上下文的查询，不适用于普通无事务查询的默认包装。

```

public OrderDetailDTO queryOrderDetail(Long orderId) {
    return transactionExecutor.readOnly("queryOrderDetail", () -> {
        OrderDO order = orderMapper.selectById(orderId);
        if (order == null) {
            throw new BizException("订单不存在");
        }
        List<OrderItemDO> items = orderItemMapper.selectList(
            new LambdaQueryWrapper<OrderItemDO>()
                .eq(OrderItemDO::getOrderId, orderId)
        );
        return buildOrderDetail(order, items);
    });
}

```

注意：

1. `readOnly` 中禁止 `insert/update/delete`。
2. 大报表、大导出不应放入只读事务。
3. 普通单表查询不必强行使用只读事务。

8.9 UnifiedTransactionExecutor 防锁表要求

使用统一事务执行器不等于自动避免锁表。锁表风险主要来自 SQL、索引、事务范围和并发设计。

事务内写 SQL 必须满足：

1. `update/delete/select for update` 必须命中索引。
2. 状态更新必须带主键或高区分度条件。
3. 状态流转必须带原状态条件。
4. 大范围更新必须按主键分批。
5. 批量 ID 必须排序后更新，降低死锁概率。
6. `for update` 优先使用主键或唯一索引。
7. 单事务锁定记录数必须可控。
8. 事务内不做慢计算、远程调用、文件处理。

推荐写法：

```

transactionExecutor.required("closeTimeoutOrders", 10, () -> {
    int updated = orderMapper.update(
        null,
        new LambdaUpdateWrapper<OrderDO>()
            .eq(OrderDO::getTenantId, tenantId)
            .gt(OrderDO::getId, startId)
            .le(OrderDO::getId, endId)
            .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
            .lt(OrderDO::getCreateTime, timeoutTime)
            .set(OrderDO::getStatus, OrderStatus.CLOSED.name())
            .set(OrderDO::getUpdateTime, LocalDateTime.now())
    );
    if (updated > 1000) {
        throw new BizException("单批更新数量超过限制");
    }
});

```

8.10 PlatformTransactionManager 定位

`PlatformTransactionManager` 是 Spring 事务体系的底层事务管理接口, `TransactionTemplate` 和 `UnifiedTransactionExecutor` 都依赖它完成事务开启、提交、回滚。

原则:

1. 普通业务代码不直接使用 `PlatformTransactionManager`。
2. 框架封装、基础设施代码、特殊事务控制场景才允许直接使用。
3. 直接使用时必须手动保证所有异常路径都会 rollback。
4. 直接使用时必须避免 commit、rollback 重复调用。

8.10.1 PlatformTransactionManager 使用边界

可以直接使用的场景:

1. 编写 `UnifiedTransactionExecutor` 这类基础设施封装。
2. 需要非常底层地控制事务定义。
3. 需要根据运行时条件动态选择传播行为、隔离级别、事务管理器。
4. 多事务管理器场景中, 需要显式选择某一个事务管理器。
5. 与框架扩展、拦截器、基础组件集成时, 无法方便使用 `TransactionTemplate`。

普通业务不推荐直接使用的原因:

1. 样板代码多, 容易漏写 rollback。
2. `commit` 前抛异常、`rollback` 又抛异常时处理复杂。
3. 容易出现事务已完成重复 rollback。
4. 可读性不如 `UnifiedTransactionExecutor`。
5. 容易把事务控制细节散落到业务代码中。

必须遵守的规则:

1. `getTransaction` 后必须保证最终进入 `commit` 或 `rollback`。
2. `commit` 和 `rollback` 只能执行一次。
3. `catch RuntimeException`、`Exception`、`Error` 都必须回滚。
4. 回滚前要判断 `status.isCompleted()`。
5. 事务定义必须设置超时时间。
6. 多数据源必须显式使用正确的 `PlatformTransactionManager`。
7. 不允许在事务内执行远程调用、MQ、缓存、文件 IO。

8.10.2 UnifiedTransactionExecutor 使用场景

业务代码默认使用 `UnifiedTransactionExecutor`。

推荐场景:

1. 接口请求中需要多表写入原子提交。
2. 消息消费中需要“幂等记录 + 业务状态更新”同事务提交。
3. 定时任务中需要按批处理数据库写入。
4. 订单、库存、账户、状态机等普通业务状态流转。
5. 业务数据和本地消息表同事务写入。
6. 事务提交后执行短任务, 例如删除缓存、轻量通知、记录非关键日志。

不适合场景:

1. 跨多个独立数据源的强一致写入。
2. 长时间任务编排。
3. 大文件处理、大报表、大批量导入不拆批。
4. 需要等待外部系统响应才能决定提交或回滚的流程。

5. 必须送达的 MQ 只靠 `afterCommit` 内存回调。

8.10.3 PlatformTransactionManager 使用场景

`PlatformTransactionManager` 只建议用于基础设施层或特殊底层场景。

推荐场景：

1. 编写 `UnifiedTransactionExecutor`、事务工具类、框架封装。
2. 需要按运行时条件动态选择事务管理器。
3. 多数据源项目中需要明确绑定某个 `PlatformTransactionManager`。
4. 需要完全手动控制 `TransactionDefinition` 和 `TransactionStatus`。
5. 框架扩展、拦截器、底层组件无法方便使用 `TransactionTemplate` 的场景。

不推荐场景：

1. 普通 service 业务方法。
2. 简单增删改查。
3. 普通多表写入。
4. 团队成员容易复制粘贴遗漏 `rollback` 的业务代码。
5. 已经可以通过 `UnifiedTransactionExecutor` 表达清楚的事务。

8.11 PlatformTransactionManager 使用 Demo

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;

@Service
public class ManualTransactionDemoService {

    private final PlatformTransactionManager transactionManager;
    private final OrderMapper orderMapper;
    private final OrderStatusLogMapper orderStatusLogMapper;

    public ManualTransactionDemoService(
        PlatformTransactionManager transactionManager,
        OrderMapper orderMapper,
        OrderStatusLogMapper orderStatusLogMapper) {
        this.transactionManager = transactionManager;
        this.orderMapper = orderMapper;
        this.orderStatusLogMapper = orderStatusLogMapper;
    }

    public void closeOrder(Long orderId) {
        DefaultTransactionDefinition definition = new
DefaultTransactionDefinition();
        definition.setName("closeOrder");

        definition.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);

        definition.setIsolationLevel(TransactionDefinition.ISOLATION_READ_COMMITTED);
        definition.setTimeout(10);
    }
}
```

```

        TransactionStatus status =
transactionManager.getTransaction(definition);
        try {
            int updated = orderMapper.update(
                null,
                new LambdaUpdateWrapper<OrderDO>()
                    .eq(OrderDO::getId, orderId)
                    .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
                    .set(OrderDO::getStatus, OrderStatus.CLOSED.name())
                    .set(OrderDO::getUpdateTime, LocalDateTime.now())
            );
            if (updated != 1) {
                throw new BizException("订单状态已变化");
            }

            OrderStatusLogDO log = new OrderStatusLogDO();
            log.setOrderId(orderId);
            log.setFromStatus(OrderStatus.CREATED.name());
            log.setToStatus(OrderStatus.CLOSED.name());
            log.setOperateTime(LocalDateTime.now());
            orderStatusLogMapper.insert(log);

            transactionManager.commit(status);
        } catch (RuntimeException ex) {
            rollbackQuietly(status);
            throw ex;
        } catch (Error error) {
            rollbackQuietly(status);
            throw error;
        } catch (Exception ex) {
            rollbackQuietly(status);
            throw new BizException("关闭订单失败", ex);
        }
    }

    private void rollbackQuietly(TransactionStatus status) {
        if (status == null || status.isCompleted()) {
            return;
        }
        transactionManager.rollback(status);
    }
}

```

8.12 UnifiedTransactionExecutor 与 PlatformTransactionManager 对比

对比项	UnifiedTransactionExecutor	PlatformTransactionManager
定位	业务层统一事务入口	Spring 底层事务管理接口
推荐使用者	业务开发	基础设施和框架封装开发
提交/回滚	模板统一托管	调用方手动控制
异常处理	已统一包装和回滚	调用方必须完整编写
代码量	少	多
出错概率	低	高
灵活性	中高	最高
慢事务日志	内置	需要自行实现
afterCommit	内置	需要自行注册同步器

结论：

1. 业务代码默认使用 `UnifiedTransactionExecutor`。
2. 不建议业务代码直接使用 `PlatformTransactionManager`。
3. `PlatformTransactionManager` 主要用于封装统一事务工具或特殊底层场景。
4. 如果发现多个业务类都在直接写 `getTransaction/commit/rollback`，应改造为 `UnifiedTransactionExecutor`。

9. MyBatis Plus 事务内代码规范

9.1 Insert 规范

1. 插入前必须完成参数校验。
2. 依赖数据库生成主键时，插入后才能使用 `entity.getId()`。
3. 唯一性必须由数据库唯一索引兜底。
4. 不允许只通过事务前查询判断唯一性。

推荐：

```
UserDO user = new UserDO();
user.setUserCode(command.getUserCode());
user.setUserName(command.getUserName());
userMapper.insert(user);
Long userId = user.getId();
```

9.2 Update 规范

更新必须带明确条件。状态流转必须带当前状态条件。

推荐：

```
int updated = orderMapper.update(
    null,
    new LambdaUpdateWrapper<OrderDO>()
        .eq(OrderDO::getId, orderId)
        .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
        .set(OrderDO::getStatus, OrderStatus.PAID.name())
        .set(OrderDO::getUpdateTime, LocalDateTime.now())
);
if (updated != 1) {
    throw new BizException("订单状态已变化");
}
```

禁止：

```
OrderDO order = orderMapper.selectById(orderId);
order.setStatus(OrderStatus.PAID.name());
orderMapper.updateById(order);
```

原因：

1. 并发情况下可能覆盖其他线程已修改的数据。
2. 状态流转没有校验原状态。
3. 如果实体字段不完整，可能导致非预期字段更新。

9.3 Delete 规范

本节只规范物理删除。逻辑删除属于业务状态控制，由具体业务模型决定，不在本节强制约束。

物理删除必须满足：

1. 删除条件必须明确，必须命中索引。
2. 禁止无条件删除。
3. 禁止按低区分度字段大范围删除。
4. 删除后必须检查影响行数。
5. 大批量删除必须按主键分批，每批独立事务。
6. 业务主表、流水表、审计表原则上不做物理删除，除非经过专项评审。

单条物理删除推荐：

```
int deleted = orderItemMapper.delete(
    new LambdaQueryWrapper<OrderItemDO>()
        .eq(OrderItemDO::getId, itemId)
        .eq(OrderItemDO::getOrderId, orderId)
);
if (deleted != 1) {
    throw new BizException("数据不存在或已被删除");
}
```

物理删除只允许用于：

- 临时表。
- 中间表。
- 明确无审计要求的数据。
- 单元测试数据清理。

批量物理删除必须分批：

```

List<Long> sortedIds = ids.stream()
    .distinct()
    .sorted()
    .collect(Collectors.toList());

List<List<Long>> partitions = ListUtils.partition(sortedIds, 500);
for (List<Long> partition : partitions) {
    transactionExecutor.required("deleteOrderItems", () -> {
        int deleted = orderItemMapper.delete(
            new LambdaQueryWrapper<OrderItemDO>()
                .eq(OrderItemDO::getOrderId, orderId)
                .in(OrderItemDO::getId, partition)
        );
        if (deleted != partition.size()) {
            throw new BizException("删除数量不符合预期");
        }
    });
}
}

```

9.4 Select 规范

事务内查询必须明确目的：

1. 校验数据是否存在。
2. 获取后续写入所需字段。
3. 锁定行。
4. 判断状态。

禁止在事务内执行大范围分页查询或报表查询。

如果需要锁定行，必须使用明确 SQL。

示例：

```

public interface OrderMapper extends BaseMapper<OrderDO> {

    @Select("select * from t_order where id = #{id} for update")
    OrderDO selectByIdForUpdate(@Param("id") Long id);

}

```

注意：

1. `for update` 必须在事务内使用才有效。
2. 锁定查询必须走索引。
3. 禁止无条件或大范围 `for update`。
4. 锁定顺序必须固定，避免死锁。

9.5 SaveBatch 规范

批量写入必须控制数量。

推荐：

```
txExecutor.requiredWithoutResult(() -> {
    List<OrderItemDO> items = buildItems(command);
    for (OrderItemDO item : items) {
        orderItemMapper.insert(item);
    }
});
```

如果使用 MyBatis Plus `saveBatch`：

```
orderItemService.saveBatch(items, 500);
```

要求：

1. 单批建议不超过 500 到 1000 条。
2. 批量数据必须提前校验。
3. 批量事务不能无限扩大。
4. 大批量导入应拆分为多个事务批次，并记录导入批次状态。

10. 异常处理规范

10.1 事务内必须抛出异常

正确：

```
txExecutor.requiredWithoutResult(() -> {
    int updated = orderMapper.updateById(order);
    if (updated != 1) {
        throw new BizException("更新失败");
    }
});
```

错误：

```
txExecutor.requiredWithoutResult(() -> {
    try {
        orderMapper.updateById(order);
    } catch (Exception e) {
        log.error("更新失败", e);
    }
});
```

10.2 不允许用返回码替代异常回滚

错误：

```
Boolean success = transactionTemplate.execute(status -> {
    int updated = orderMapper.updateById(order);
    if (updated != 1) {
        return false;
    }
    return true;
});
```

正确：

```
Long orderId = transactionTemplate.execute(status -> {
    int updated = orderMapper.updateById(order);
    if (updated != 1) {
        throw new BizException("更新失败");
    }
    return order.getId();
});
```

10.3 需要转义异常时必须保留 cause

```
try {
    txExecutor.requiredWithoutResult(() -> saveData(command));
} catch (DuplicateKeyException e) {
    throw new BizException("编码已存在", e);
} catch (DataAccessException e) {
    throw new BizException("数据库操作失败", e);
}
```

11. 事务范围规范

11.1 事务外执行

以下逻辑应放在事务外：

- 参数基础校验。
- DTO 转换。
- 不影响一致性的普通查询。
- 远程服务查询。
- 文件解析。
- Excel 读取。
- 图片处理。
- 消息体组装。
- 返回对象组装。

11.2 事务内执行

以下逻辑应放在事务内：

- 互相依赖的多表写入。
- 状态流转和状态流水写入。
- 扣库存、加库存。
- 账户余额变更。
- 幂等记录插入。
- 本地消息表写入。
- 需要原子提交的审计日志。

11.3 事务提交后执行

以下逻辑应在事务提交后执行：

- 发送 MQ。
- 调用外部系统。

- 删除缓存。
- 推送通知。
- 触发异步任务。

推荐使用本地消息表：

```
txExecutor.requiredWithoutResult(() -> {  
    orderMapper.insert(order);  
    localMessageMapper.insert(buildOrderCreatedMessage(order));  
});  
  
messagePublishJob.publishPendingMessages();
```

12. 并发控制规范

12.1 优先使用条件更新

库存扣减推荐：

```
int updated = productMapper.update(  
    null,  
    new LambdaUpdateWrapper<ProductDO>()  
        .eq(ProductDO::getId, productId)  
        .ge(ProductDO::getStock, quantity)  
        .setSql("stock = stock - " + quantity)  
);  
if (updated != 1) {  
    throw new BizException("库存不足");  
}
```

12.2 状态机必须校验原状态

```
int updated = taskMapper.update(  
    null,  
    new LambdaUpdateWrapper<TaskDO>()  
        .eq(TaskDO::getId, taskId)  
        .eq(TaskDO::getStatus, TaskStatus.WAITING.name())  
        .set(TaskDO::getStatus, TaskStatus.RUNNING.name())  
);  
if (updated != 1) {  
    throw new BizException("任务状态已变化");  
}
```

12.3 乐观锁规范

如果表中有 `version` 字段，应启用 MyBatis Plus 乐观锁插件。

```
@Version  
private Integer version;
```

更新时必须检查返回行数：

```
int updated = orderMapper.updateById(order);
if (updated != 1) {
    throw new BizException("数据已被其他人修改");
}
```

12.4 悲观锁规范

使用 `for update` 时必须满足：

1. 查询条件命中索引。
2. 加锁顺序固定。
3. 锁范围尽量小。
4. 事务时间尽量短。
5. 不允许锁表扫描。

13. 幂等规范

外部请求、消息消费、定时补偿必须设计幂等。

13.1 唯一键幂等

表设计：

```
create unique index uk_request_id on t_order_request(request_id);
```

代码示例：

```
txExecutor.requiredWithoutResult(() -> {
    try {
        OrderRequestDO request = new OrderRequestDO();
        request.setRequestId(command.getRequestId());
        request.setCreateTime(LocalDateTime.now());
        orderRequestMapper.insert(request);
    } catch (DuplicateKeyException e) {
        throw new BizException("请求已处理", e);
    }

    orderMapper.insert(order);
});
```

13.2 消息消费幂等

```
public void consume(OrderPaidMessage message) {
    txExecutor.requiredWithoutResult(() -> {
        try {
            MessageConsumedDO consume = new MessageConsumedDO();
            consume.setMessageId(message.getMessageId());
            consume.setCreateTime(LocalDateTime.now());
            messageConsumeMapper.insert(consume);
        } catch (DuplicateKeyException e) {
            return;
        }

        int updated = orderMapper.update(
```

```

        null,
        new LambdaUpdateWrapper<OrderDO>()
            .eq(OrderDO::getId, message.getOrderId())
            .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
            .set(OrderDO::getStatus, OrderStatus.PAID.name())
    );
    if (updated != 1) {
        throw new BizException("订单状态不允许支付");
    }
    });
}

```

注意：

1. 消费记录和业务处理必须在同一事务内。
2. 重复消息可以直接返回成功。
3. 不能先执行业务再插入消费记录。

14. 多数据源规范

14.1 单事务只控制一个数据源

普通 Spring 本地事务只能保证同一个数据源内的事务一致性。

禁止：

```

txExecutor.requiredWithoutResult(() -> {
    db1UserMapper.insert(user);
    db2LogMapper.insert(log);
});

```

除非明确使用了支持分布式事务的方案，否则上述代码不能保证两个数据源同时提交或同时回滚。

14.2 多事务管理器必须显式区分

```

@Bean
public TxExecutor orderTxExecutor(
    @Qualifier("orderTransactionManager") PlatformTransactionManager
    transactionManager) {
    return new TxExecutor(transactionManager);
}

@Bean
public TxExecutor logTxExecutor(
    @Qualifier("logTransactionManager") PlatformTransactionManager
    transactionManager) {
    return new TxExecutor(transactionManager);
}

```

使用时按数据源选择对应执行器。

15. 事务与 MQ 规范

15.1 禁止事务内直接发送 MQ

错误:

```
txExecutor.requiredWithoutResult(() -> {
    orderMapper.insert(order);
    mqProducer.send(orderCreatedMessage);
});
```

风险:

1. MQ 发送成功后数据库回滚, 会产生脏消息。
2. 数据库提交成功后 MQ 发送失败, 会丢失通知。
3. 事务内发送 MQ 会拉长事务时间。

15.2 推荐本地消息表

```
txExecutor.requiredWithoutResult(() -> {
    orderMapper.insert(order);

    LocalMessageDO message = new LocalMessageDO();
    message.setMessageId(IdGenerator.nextId());
    message.setTopic("order.created");
    message.setBody(JsonUtils.toJson(order));
    message.setStatus(MessageStatus.PENDING.name());
    message.setCreateTime(LocalDateTime.now());
    localMessageMapper.insert(message);
});
```

然后由独立任务扫描 PENDING 消息发送。

发送成功后:

```
int updated = localMessageMapper.update(
    null,
    new LambdaUpdateWrapper<LocalMessageDO>()
        .eq(LocalMessageDO::getMessageId, messageId)
        .eq(LocalMessageDO::getStatus, MessageStatus.PENDING.name())
        .set(LocalMessageDO::getStatus, MessageStatus.SENT.name())
        .set(LocalMessageDO::getUpdateTime, LocalDateTime.now())
);
if (updated != 1) {
    throw new BizException("消息状态已变化");
}
```

16. 事务与缓存规范

16.1 禁止事务内删除缓存

错误:

```
txExecutor.requiredWithoutResult(() -> {
    orderMapper.updateById(order);
    redisTemplate.delete("order:" + order.getId());
});
```

风险：

1. 缓存删除成功后事务回滚，缓存与数据库短暂不一致。
2. Redis 调用会拉长数据库事务。

16.2 推荐提交后处理缓存

简单场景：

```
txExecutor.requiredWithoutResult(() -> {  
    orderMapper.updateById(order);  
});  
  
redisTemplate.delete("order:" + order.getId());
```

关键场景推荐使用本地消息表或延迟双删策略。

17. 事务与异步规范

事务内禁止启动异步线程访问未提交数据。

错误：

```
txExecutor.requiredWithoutResult(() -> {  
    orderMapper.insert(order);  
    asyncService.notify(order.getId());  
});
```

正确：

```
Long orderId = txExecutor.required(() -> {  
    orderMapper.insert(order);  
    return order.getId();  
});  
  
asyncService.notify(orderId);
```

如果异步任务必须保证不丢，应写入本地任务表。

18. 事务超时规范

1. 所有写事务必须设置超时时间。
2. 普通业务写事务建议 5 到 10 秒。
3. 批量导入单批事务建议 10 到 30 秒。
4. 禁止无限超时。
5. 超时后必须回滚并记录业务主键、请求号、批次号。

示例：

```
TransactionTemplate template = new TransactionTemplate(transactionManager);  
template.setTimeout(10);
```

19. 隔离级别规范

默认推荐 `READ_COMMITTED`。

隔离级别	使用建议
<code>READ_UNCOMMITTED</code>	禁止使用，可能脏读
<code>READ_COMMITTED</code>	默认推荐
<code>REPEATABLE_READ</code>	MySQL 默认，可用于需要可重复读的场景
<code>SERIALIZABLE</code>	谨慎使用，容易降低并发

注意：

1. 不要通过提高隔离级别掩盖并发设计问题。
2. 库存、余额、状态流转优先使用条件更新或锁。
3. 隔离级别必须结合具体数据库行为评估。

20. 日志规范

事务失败日志必须包含：

- 业务动作。
- 业务主键。
- 请求号或 `traceId`。
- 操作人。
- 异常堆栈。

示例：

```
try {
    txExecutor.requiredWithoutResult(() -> cancelOrderInTx(orderId,
operatorId));
} catch (Exception e) {
    log.error(
        "cancel order failed, orderId={}, operatorId={}, traceId={}",
        orderId,
        operatorId,
        TraceContext.getTraceId(),
        e
    );
    throw e;
}
```

禁止：

```
log.error("操作失败");
```

21. 测试规范

21.1 必测场景

涉及事务的代码必须覆盖：

1. 正常提交。

2. 中间步骤异常后全部回滚。
3. 受检异常包装后回滚。
4. 并发更新失败。
5. 重复请求幂等。
6. 唯一键冲突。
7. 状态不匹配。
8. 批量部分失败回滚或按批提交。

21.2 回滚测试示例

```
@Test
void shouldRollbackWhenCreateOrderItemFailed() {
    CreateOrderCommand command = new CreateOrderCommand();
    command.setUserId(1L);
    command.setProductId(100L);
    command.setQuantity(1);

    assertThrows(BizException.class, () ->
        orderApplicationService.createOrder(command));

    Long orderCount = orderMapper.selectCount(
        new LambdaQueryWrapper<OrderDO>()
            .eq(OrderDO::getProductId, 100L)
    );
    assertEquals(0L, orderCount);
}
```

21.3 测试注意事项

1. 测试不能因为测试框架自己的事务回滚而掩盖业务事务问题。
2. 如果测试方法上使用了 `@Transactional`，需要特别注意它会影响被测代码事务边界。
3. 推荐使用真实数据库或 Testcontainers 验证事务行为。
4. 并发问题需要用多线程测试或集成测试验证。

22. 代码审查清单

提交事务相关代码时，必须检查以下项目：

检查项	要求
是否使用 <code>@Transactional</code>	新代码不推荐
是否使用手动事务	必须
事务范围是否最小	必须
是否设置超时时间	必须
异常是否会触发回滚	必须
是否吞异常	禁止
是否事务内远程调用	禁止
是否事务内发送 MQ	禁止
是否事务内删除缓存	原则上禁止
更新是否检查返回行数	必须
状态流转是否校验原状态	必须
库存余额是否使用条件更新	必须
是否有幂等设计	外部请求和消息消费必须
是否存在长事务	禁止
是否跨数据源	必须专项评审
是否有回滚测试	必须

23. 常见错误示例

23.1 错误：事务内调用远程接口

```
txExecutor.requiredWithoutResult(() -> {
    orderMapper.insert(order);
    paymentClient.freezeAmount(order.getId());
});
```

问题：

1. 远程调用慢会导致长事务。
2. 远程调用成功后本地事务可能回滚。
3. 远程调用失败时本地数据库是否回滚不一定符合业务补偿要求。

23.2 错误：先查库存再更新

```
ProductDO product = productMapper.selectById(productId);
if (product.getStock() < quantity) {
    throw new BizException("库存不足");
}
product.setStock(product.getStock() - quantity);
productMapper.updateById(product);
```

问题：

并发情况下多个事务可能读到相同库存，导致超卖。

23.3 错误：状态更新不带原状态

```
orderMapper.update(  
    null,  
    new LambdaUpdateWrapper<OrderDO>()  
        .eq(OrderDO::getId, orderId)  
        .set(OrderDO::getStatus, OrderStatus.PAID.name())  
);
```

问题：

取消状态、关闭状态、退款状态都可能被错误覆盖成已支付。

23.4 错误：事务内循环处理大量数据

```
txExecutor.requiredWithoutResult(() -> {  
    for (ImportRow row : rows) {  
        mapper.insert(convert(row));  
    }  
});
```

如果 `rows` 很大，会导致长事务、连接长期占用、锁长期持有。大批量导入必须分批处理。

24. 推荐分层规范

推荐分层：

```
Controller  
-> ApplicationService  
    -> TxExecutor.required(...)  
        -> Mapper / DomainService  
            -> MQ / Cache / Remote after commit
```

职责说明：

层级	职责
Controller	参数接收、基础鉴权、返回结果
ApplicationService	编排业务流程、控制事务边界
DomainService	领域规则、纯业务判断
Mapper	数据库访问
TxExecutor	统一事务执行
MessageJob	事务后消息发送、补偿

事务边界应放在 `ApplicationService`，不要下沉到 `Mapper`，也不要放在 `Controller`。

25. 标准模板

25.1 有返回值事务模板

```
public ResultDTO executeBusiness(Command command) {
    validate(command);
    prepareOutsideTransaction(command);

    Long id = txExecutor.required(() -> {
        int updated = mapper.update(
            null,
            new LambdaUpdateWrapper<EntityDO>()
                .eq(EntityDO::getId, command.getId())
                .eq(EntityDO::getStatus, Status.INIT.name())
                .set(EntityDO::getStatus, Status.SUCCESS.name())
        );
        if (updated != 1) {
            throw new BizException("状态已变化");
        }

        LogDO log = buildLog(command);
        logMapper.insert(log);
        return command.getId();
    });

    notifyAfterTransaction(id);
    return buildResult(id);
}
```

25.2 无返回值事务模板

```
public void executeBusiness(Command command) {
    validate(command);

    txExecutor.requiredWithoutResult(() -> {
        EntityDO entity = buildEntity(command);
        mapper.insert(entity);

        AuditLogDO log = buildAuditLog(entity);
        auditLogMapper.insert(log);
    });
}
```

25.3 批量分批事务模板

```
public void importData(List<ImportRow> rows) {
    validateRows(rows);

    List<List<ImportRow>> partitions = ListUtils.partition(rows, 500);
    for (List<ImportRow> partition : partitions) {
        txExecutor.requiredWithoutResult(() -> {
            for (ImportRow row : partition) {
                mapper.insert(convert(row));
            }
        });
    }
}
```

26. MySQL 锁表风险与注意事项

本章节中的“锁表”包含两类情况：

1. 真正的表级锁，例如 `lock table`、DDL 元数据锁、部分表锁。
2. 业务上表现为“像锁表一样”的严重锁等待，例如无索引更新导致大量行锁、间隙锁阻塞插入、长事务持锁不释放。

MySQL InnoDB 正常情况下以行锁为主，但行锁必须依赖索引命中。如果 SQL 写法、索引设计或事务边界不合理，行锁会扩大成大范围锁等待，最终表现为接口卡死、连接池耗尽、更新超时、DDL 无法执行。

26.1 锁表高风险操作清单

以下操作必须重点评审：

操作	风险
无索引条件 <code>update</code>	扫描大量记录并加锁，表现为锁表
无索引条件 <code>delete</code>	扫描大量记录并加锁，严重阻塞其他写入
范围条件 <code>update/delete</code>	可能产生大量行锁和间隙锁
<code>select ... for update</code>	命中范围过大时会阻塞大量事务
长事务内更新热点数据	锁长时间不释放
事务内远程调用	外部耗时放大锁持有时间
大批量单事务导入	长时间占用锁、undo、redo 和连接
DDL 操作	可能等待或阻塞元数据锁
在线改表工具使用不当	可能放大主从延迟或产生元数据锁等待
<code>lock tables</code>	直接表锁，业务代码禁止
<code>alter table</code>	获取元数据锁，可能阻塞查询和写入
<code>truncate table</code>	DDL 操作，隐式提交并获取元数据锁

26.2 坑 1：更新条件没有索引

错误示例：

```
update t_order
set status = 'CLOSED'
where request_no = 'REQ_001';
```

如果 `request_no` 没有索引，MySQL 需要扫描大量记录。InnoDB 会对扫描过程中匹配或访问到的记录加锁，实际效果可能接近锁表。

MyBatis Plus 错误示例：

```
orderMapper.update(
    null,
    new LambdaUpdateWrapper<OrderDO>()
        .eq(OrderDO::getRequestNo, requestNo)
        .set(OrderDO::getStatus, OrderStatus.CLOSED.name())
);
```

规范要求：

1. `update/delete/select for update` 的 `where` 条件必须命中索引。
2. 高频更新字段必须建立合适索引。
3. 上线前必须执行 `explain`。
4. 影响行数必须符合预期。

推荐：

```
create unique index uk_order_request_no on t_order(request_no);
```

26.3 坑 2：索引字段被函数处理导致索引失效

错误示例：

```
update t_order
set status = 'EXPIRED'
where date(create_time) = '2026-07-09';
```

`date(create_time)` 会导致普通 `create_time` 索引失效，可能扫描大量数据并加锁。

推荐：

```
update t_order
set status = 'EXPIRED'
where create_time >= '2026-07-09 00:00:00'
    and create_time < '2026-07-10 00:00:00';
```

规范要求：

1. 禁止在更新和删除条件中对索引列使用函数。
2. 禁止对索引列做表达式计算。
3. 时间范围必须使用左闭右开区间。

26.4 坑 3：隐式类型转换导致索引失效

错误示例：

```
update t_user
set status = 'DISABLED'
where phone = 13800138000;
```

如果 `phone` 是 `varchar`，但 SQL 使用数字条件，MySQL 可能发生隐式类型转换，导致索引失效。

正确示例：

```
update t_user
set status = 'DISABLED'
where phone = '13800138000';
```

规范要求：

1. Java 参数类型必须与数据库字段类型一致。
2. 字符串字段条件必须传字符串。
3. 禁止用 `apply` 拼接未校验 SQL。

26.5 坑 4：最左前缀不匹配

表索引：

```
create index idx_tenant_status_time on t_task(tenant_id, status, create_time);
```

错误示例：

```
update t_task
set status = 'TIMEOUT'
where status = 'RUNNING'
and create_time < '2026-07-09 10:00:00';
```

该 SQL 跳过了联合索引第一列 `tenant_id`，可能无法有效使用索引。

正确示例：

```
update t_task
set status = 'TIMEOUT'
where tenant_id = 1001
and status = 'RUNNING'
and create_time < '2026-07-09 10:00:00';
```

规范要求：

1. 联合索引条件必须尽量满足最左前缀。
2. 多租户表更新必须带 `tenant_id`。
3. 定时任务扫描多租户数据时必须按租户或主键分批。

26.6 坑 5：范围更新过大

错误示例：

```
update t_log
set deleted = 1
where create_time < '2026-01-01 00:00:00';
```

即使命中索引，如果范围过大，也会锁住大量记录，并产生大量 undo/redo。

推荐分批：

```
update t_log
set deleted = 1
where id > 100000
      and id <= 101000
      and create_time < '2026-01-01 00:00:00';
```

规范要求：

1. 大范围更新必须按主键分批。
2. 每批建议 500 到 2000 行，按业务压力调整。
3. 每批单独事务提交。
4. 批处理任务必须支持断点续跑。

26.7 坑 6: delete 大量数据

错误示例：

```
delete from t_request_log
where create_time < '2026-01-01 00:00:00';
```

风险：

1. 长时间持有大量行锁。
2. 产生大量 undo/redo。
3. 主从复制延迟。
4. 回滚成本极高。
5. 可能导致业务写入等待。

推荐：

1. 先逻辑删除，再异步物理清理。
2. 按主键分批删除。
3. 低峰期执行。
4. 每批提交。
5. 归档表优先使用分区或按月分表。

26.8 坑 7: select for update 没有命中唯一索引

错误示例：

```
select *
from t_order
where status = 'CREATED'
for update;
```

该 SQL 会锁住大量 CREATED 状态订单，并可能阻塞新订单状态变更。

推荐：

```
select *
from t_order
where id = 1001
for update;
```

或：

```
select *
from t_order
where order_no = '0202607090001'
for update;
```

前提是 `id` 或 `order_no` 是唯一索引。

规范要求：

1. `for update` 优先使用主键或唯一索引。
2. 非唯一索引 `for update` 必须确认锁范围。
3. 范围 `for update` 必须专项评审。
4. `for update` 必须在事务中使用，否则锁会立即释放或行为不符合预期。

26.9 坑 8：间隙锁阻塞插入

在 MySQL InnoDB 的 `REPEATABLE READ` 隔离级别下，范围查询、范围更新、`for update` 可能产生 next-key lock，也就是记录锁加间隙锁。

示例：

```
select *
from t_order
where order_no > '0100'
      and order_no < '0200'
for update;
```

该 SQL 不只锁定已有记录，还可能锁住范围间隙，导致其他事务无法插入 `0100` 到 `0200` 之间的新记录。

注意事项：

1. 范围锁可能阻塞插入，不只是阻塞更新。
2. 唯一索引等值查询通常锁范围更小。
3. 默认隔离级别如果是 `REPEATABLE READ`，要特别关注间隙锁。
4. 业务默认推荐 `READ_COMMITTED`，可以减少部分间隙锁问题，但不能替代索引和 SQL 设计。

26.10 坑 9：非唯一索引等值查询锁住多行和间隙

表索引：

```
create index idx_status on t_task(status);
```

SQL：

```
select *
from t_task
where status = 'WAITING'
for update;
```

因为 `status` 不是唯一索引，所有 `WAITING` 记录都可能被锁住，且在 `REPEATABLE READ` 下还可能锁住相邻间隙。

规范要求：

1. 不要用低区分度字段做锁定条件。
2. 状态类字段不能单独作为 `for update` 条件。
3. 任务抢占应使用主键分批扫描加条件更新。

任务抢占推荐：

```
int updated = taskMapper.update(
    null,
    new LambdaUpdateWrapper<TaskDO>()
        .eq(TaskDO::getId, taskId)
        .eq(TaskDO::getStatus, TaskStatus.WAITING.name())
        .set(TaskDO::getStatus, TaskStatus.RUNNING.name())
        .set(TaskDO::getWorkerId, workerId)
);
if (updated != 1) {
    return;
}
```

26.11 坑 10：长事务导致锁长期不释放

错误示例：

```
txExecutor.requiredWithoutResult(() -> {
    orderMapper.updateById(order);
    remotePaymentClient.freeze(order.getId());
    fileService.upload(file);
    messageProducer.send(message);
});
```

问题：

1. 数据库锁直到事务提交才释放。
2. 外部调用越慢，锁持有越久。
3. 连接池连接也被长时间占用。
4. 并发请求会堆积，最终雪崩。

规范要求：

1. 事务内禁止远程调用。
2. 事务内禁止文件 IO。
3. 事务内禁止发送 MQ。
4. 事务内禁止长时间循环。
5. 事务代码必须只包含必要数据库操作。

26.12 坑 11：事务开启后先查询再慢处理

错误示例：

```
txExecutor.requiredWithoutResult(() -> {  
    OrderDO order = orderMapper.selectByIdForUpdate(orderId);  
    BigDecimal amount = complexCalculate(order);  
    orderMapper.updateById(order);  
});
```

如果 `complexCalculate` 很慢，锁会一直被持有。

推荐：

```
OrderDO orderSnapshot = orderMapper.selectById(orderId);  
BigDecimal amount = complexCalculate(orderSnapshot);  
  
txExecutor.requiredWithoutResult(() -> {  
    OrderDO lockedOrder = orderMapper.selectByIdForUpdate(orderId);  
    checkOrderCanUpdate(lockedOrder);  
    lockedOrder.setAmount(amount);  
    orderMapper.updateById(lockedOrder);  
});
```

原则：

1. 慢计算放事务外。
2. 事务内重新校验关键状态。
3. 加锁后快速更新并提交。

26.13 坑 12：不同事务加锁顺序不一致导致死锁

事务 A：

先锁订单，再锁账户

事务 B：

先锁账户，再锁订单

两个事务可能互相等待，产生死锁。

规范要求：

1. 同一业务链路必须固定加锁顺序。
2. 多行加锁必须按主键升序。
3. 多表加锁必须有统一顺序。
4. 死锁异常必须允许重试，但不能无限重试。

推荐顺序：

租户 -> 用户 -> 账户 -> 订单 -> 订单明细 -> 流水

具体项目可按业务模型制定统一顺序。

26.14 坑 13：批量更新未排序导致死锁

错误示例：

```
for (Long orderId : orderIds) {
    orderMapper.update(
        null,
        new LambdaUpdateWrapper<OrderDO>()
            .eq(OrderDO::getId, orderId)
            .set(OrderDO::getStatus, OrderStatus.CLOSED.name())
    );
}
```

如果不同线程传入的 `orderIds` 顺序不同，就可能交叉加锁。

推荐：

```
List<Long> sortedOrderIds = orderIds.stream()
    .distinct()
    .sorted()
    .toList();

for (Long orderId : sortedOrderIds) {
    orderMapper.update(
        null,
        new LambdaUpdateWrapper<OrderDO>()
            .eq(OrderDO::getId, orderId)
            .set(OrderDO::getStatus, OrderStatus.CLOSED.name())
    );
}
```

26.15 坑 14：热点行高并发更新

典型热点：

- 库存总量行。
- 账户余额行。
- 全局配置行。
- 统计计数行。
- 序列表。

错误示例：

```
update t_counter
set value = value + 1
where code = 'GLOBAL';
```

高并发下所有请求都竞争同一行锁。

优化方向：

1. 分片计数。
2. Redis 原子计数加异步落库。
3. 按业务维度拆分库存。
4. 使用流水表追加写，异步汇总。
5. 降低单行更新频率。

26.16 坑 15： `updateById` 覆盖字段导致锁时间和冲突扩大

如果实体包含大量字段，`updateById` 可能生成较多字段更新，增加行变更成本，也可能覆盖其他并发修改。

不推荐：

```
OrderDO order = orderMapper.selectById(orderId);
order.setStatus(OrderStatus.PAID.name());
orderMapper.updateById(order);
```

推荐：

```
orderMapper.update(
    null,
    new LambdaUpdateWrapper<OrderDO>()
        .eq(OrderDO::getId, orderId)
        .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
        .set(OrderDO::getStatus, OrderStatus.PAID.name())
        .set(OrderDO::getUpdateTime, LocalDateTime.now())
);
```

规范要求：

1. 状态更新使用 `LambdaUpdateWrapper`。
2. 只更新必要字段。
3. 必须带原状态条件。
4. 必须检查影响行数。

26.17 坑 16：MyBatis Plus `setSql` 拼接不受控

错误示例：

```
.setSql("stock = stock - " + quantity)
```

如果 `quantity` 没有经过严格校验，存在 SQL 注入或异常 SQL 风险。

要求：

1. `quantity` 必须是服务端校验后的正整数。
2. 禁止拼接用户输入字符串。
3. 能使用参数绑定时优先使用参数绑定。
4. `setSql` 只允许用于简单、可审查的原子更新表达式。

26.18 坑 17：分页扫描时同时更新扫描条件

错误示例：

```
select id from t_task where status = 'WAITING' limit 100;
update t_task set status = 'RUNNING' where id in (...);
```

多个 worker 并发执行时可能抢到同一批数据，互相等待或重复处理。

推荐：

1. 先按主键分页扫描候选 ID。
2. 对每条任务使用 `id + status` 条件更新抢占。

3. 更新成功才执行任务。
4. 避免锁住整批候选数据。

26.19 坑 18: `in` 条件过大

错误示例：

```
update t_order
set status = 'CLOSED'
where id in (... 几万条 ...);
```

MySQL 不一定直接报错，它的In取决于max_allowed_packet配置，但会带来 SQL 过长、解析慢、锁范围大、事务膨胀、死锁概率上升等问题。

风险：

1. SQL 解析成本高。
2. 锁大量行。
3. 容易死锁。
4. 回滚成本大。

规范要求：

1. `in` 条件必须分批。
2. 单批建议不超过 500 到 1000 个 ID。
3. ID 必须排序。
4. 每批单独事务。

26.20 坑 19: DDL 元数据锁阻塞业务

DDL 需要获取 metadata lock。只要有事务正在访问表，DDL 可能等待；DDL 等待期间，后续访问同一表的 SQL 也可能被阻塞。

典型事故：

```
alter table t_order add column ext_info varchar(255);
```

如果此时存在长事务查询或更新 `t_order`，`alter table` 等待元数据锁。随后新的 `select/update/insert` 也可能排队，造成业务大面积阻塞。

规范要求：

1. 生产 DDL 必须低峰执行。
2. DDL 前必须检查长事务。
3. 大表 DDL 必须评估在线 DDL 方案。
4. DDL 必须设置执行窗口和回滚预案。
5. 禁止业务高峰直接改大表。

排查 SQL：

```
show processlist;
```

```
select *
from information_schema.innodb_trx;
```

26.21 坑 20：长查询也会阻塞 DDL

即使普通 `select` 不加行锁，也会持有 metadata lock。长查询未结束时，DDL 可能无法获取元数据锁。

高风险查询：

1. 无索引大表查询。
2. 慢报表查询。
3. 导出全表数据。
4. 事务中执行查询后长时间不提交。

规范要求：

1. 报表和导出必须走只读库或离线库。
2. 禁止在事务中执行大查询。
3. DDL 前必须确认无长查询。

26.22 坑 21：lock tables 直接锁表

业务代码禁止使用：

```
lock tables t_order write;
```

原因：

1. 直接阻塞其他会话读写。
2. 容易造成连接长期持锁。
3. 与连接池结合风险极高。
4. 应用异常时排查困难。

26.23 坑 22：truncate table 是 DDL

`truncate table` 不是普通删除，而是 DDL：

1. 会隐式提交当前事务。
2. 获取元数据锁。
3. 不能按行回滚。
4. 可能阻塞其他访问。

业务代码禁止对业务表执行 `truncate`。

26.24 坑 23：外键级联导致隐式锁扩大

如果表存在外键约束，更新或删除父表记录时，MySQL 需要检查或级联子表数据，可能导致额外锁等待。

规范要求：

1. 高并发核心业务表谨慎使用外键。
2. 如果使用外键，子表外键列必须有索引。
3. 删除父表数据必须评估子表影响范围。
4. 业务层显式维护关联关系，避免不可控级联。

26.25 坑 24：唯一索引冲突也会产生锁等待

并发插入相同唯一键：

```
insert into t_request(request_id) values ('REQ_001');
```

一个事务插入后未提交，另一个事务插入相同唯一键会等待。

注意事项：

1. 幂等表插入后事务要尽快提交。
2. 幂等记录不要放在长事务开头后执行大量慢逻辑。
3. 唯一键冲突必须捕获 `DuplicateKeyException`。

26.26 坑 25：自增主键热点与插入等待

高并发插入使用自增主键时，可能受自增锁、页分裂、热点页影响。通常不是传统锁表，但会表现为插入性能下降。

优化方向：

1. 控制单表高并发写入。
2. 使用合理分库分表或分区。
3. 避免超大单表无限增长。
4. 大日志表按时间归档。

26.27 坑 26：低区分度索引导致锁范围过大

低区分度字段包括：

- `status`。
- `deleted`。
- `enabled`。
- `type`。
- `tenant_id` 在单租户大数据量场景。

错误示例：

```
update t_order
set status = 'TIMEOUT'
where status = 'CREATED';
```

即使 `status` 有索引，也可能锁住大量记录。

规范要求：

1. 低区分度索引不能单独作为大范围更新条件。
2. 必须组合主键、时间、租户、业务号等高区分度字段。
3. 定时任务必须按主键窗口推进。

26.28 坑 27：or 条件导致索引使用变差

错误示例：

```
update t_order
set status = 'CLOSED'
where order_no = '0001'
or request_no = 'R001';
```

如果其中一个字段没有索引，整体可能退化为扫描。

规范要求：

1. 更新和删除中谨慎使用 `or`。
2. `or` 两侧字段都必须有合适索引。
3. 可拆成两条明确命中索引的 SQL。

26.29 坑 28：模糊查询更新导致扫描

错误示例：

```
update t_user
set risk_flag = 1
where name like '%test%';
```

前置 `%` 通常无法使用普通 BTree 索引，可能全表扫描并加锁。

规范要求：

1. 禁止基于前置模糊条件执行更新或删除。
2. 先离线筛选主键，再按主键分批更新。
3. 搜索场景使用搜索引擎或专用索引。

26.30 坑 29：排序更新和范围锁

示例：

```
update t_task
set status = 'RUNNING'
where status = 'WAITING'
order by create_time
limit 100;
```

如果索引不匹配 `where + order by`，可能扫描、排序并锁住大量记录。

规范要求：

1. 必须有匹配索引，例如 `(status, create_time, id)`。
2. 抢占任务时必须检查执行计划。
3. 并发 worker 下仍需用状态条件判断影响行数。

26.31 坑 30：读写事务混在一起

错误示例：

```
txExecutor.requiredWithoutResult(() -> {
    List<OrderDO> orders = orderMapper.selectList(queryWrapper);
    reportService.calculate(orders);
    orderMapper.updateById(order);
});
```

问题：

1. 查询和计算拉长事务。
2. 更新锁释放变慢。
3. 查询结果可能很大，导致事务膨胀。

规范要求：

1. 查询准备阶段放事务外。
2. 事务内只做最终校验和写入。
3. 报表类逻辑禁止放入写事务。

26.32 坑 31：连接池连接被锁等待占满

当大量请求等待同一把锁时，每个请求都会占用数据库连接。连接池耗尽后，即使不访问同一张表的接口也可能失败。

规范要求：

1. 设置合理的锁等待超时。
2. 设置事务超时。
3. 热点写入必须限流。
4. 批处理任务必须限制并发。
5. 监控连接池活跃连接数和等待线程数。

26.33 坑 32：锁等待超时时间过长

MySQL 参数 `innodb_lock_wait_timeout` 如果过长，请求会长时间卡住。

建议：

1. 应用层事务超时必须小于用户可接受等待时间。
2. 关键接口要做快速失败。
3. 对可重试场景做有限次数退避重试。
4. 不允许无限等待数据库锁。

26.34 坑 33：死锁后没有重试策略

MySQL 检测到死锁时会回滚其中一个事务。应用会收到异常。

规范要求：

1. 幂等操作可做有限重试。
2. 重试必须有最大次数，建议 2 到 3 次。
3. 重试必须有随机退避。
4. 非幂等操作禁止盲目重试。
5. 死锁日志必须保留业务主键和 SQL 场景。

26.35 坑 34：批处理和在线业务抢同一批数据

批处理更新线上正在访问的数据，会造成锁竞争。

规范要求：

1. 批处理必须低峰运行。
2. 批处理必须限速。
3. 批处理必须按主键分段。
4. 批处理必须支持暂停和恢复。
5. 在线热点表批处理必须经过 DBA 或架构评审。

26.36 坑 35：缺少影响行数校验

错误示例：

```
orderMapper.update(  
    null,  
    new LambdaUpdateWrapper<OrderDO>()  
        .eq(OrderDO::getId, orderId)  
        .set(OrderDO::getStatus, OrderStatus.PAID.name())  
);
```

没有检查返回值时，可能发生：

1. 没有更新任何行但业务继续执行。
2. 更新多行但业务误以为只更新一行。
3. 并发状态冲突被忽略。

规范要求：

```
int updated = orderMapper.update(...);  
if (updated != 1) {  
    throw new BizException("数据状态已变化");  
}
```

26.37 坑 36：没有执行计划审查

以下 SQL 必须提供 `explain` 结果：

1. 生产写 SQL。
2. 批量更新 SQL。
3. 批量删除 SQL。
4. `select for update`。
5. 定时任务扫描 SQL。
6. 消息消费等查询和更新 SQL。

重点检查：

字段	要求
type	避免 <code>ALL</code>
key	必须命中预期索引
rows	扫描行数必须可控
Extra	避免大范围 <code>using filesort</code> 、 <code>Using temporary</code>

26.38 坑 37：事务中等待用户输入或人工确认

禁止在事务中执行任何需要用户、人工、外部审批、队列回调才能继续的逻辑。

错误场景：

1. 事务打开后等待审批。
2. 事务打开后等待第三方支付回调。
3. 事务打开后等待异步任务结果。

事务必须是短时间内自动完成的数据库一致性单元。

26.39 坑 38：跨库跨表误以为一个事务能全部回滚

本地事务只能保证同一个事务管理器控制的数据源。跨库操作可能出现一边提交、一边失败。

风险：

1. 数据不一致。
2. 锁等待无法统一处理。
3. 回滚语义错误。

规范要求：

1. 跨库写入必须使用最终一致方案、分布式事务方案或补偿方案。
2. 不允许用一个普通 `TransactionTemplate` 包装多个数据源写入后宣称强一致。

26.40 锁表排查 SQL

查看当前连接：

```
show processlist;
```

查看 InnoDB 事务：

```
select *  
from information_schema.innodb_trx;
```

查看锁等待关系，MySQL 8 可使用：

```
select *  
from performance_schema.data_lock_waits;
```

查看锁详情：

```
select *  
from performance_schema.data_locks;
```

查看最近一次死锁：

```
show engine innodb status;
```

排查重点：

1. 找出持锁时间最长的事务。
2. 找出正在等待的 SQL。
3. 找出阻塞源连接 ID。
4. 判断是否可以终止阻塞连接。
5. 保存现场 SQL、事务、线程、业务 traceId。

26.41 锁表事故处理原则

1. 先确认阻塞源，不要盲目重启应用。
2. 先保存 `show processlist` 和 InnoDB 状态。
3. 如果是 DDL 等待元数据锁，优先评估是否 kill DDL 或长事务。
4. 如果是业务长事务，联系业务负责人确认能否 kill。
5. kill 前必须评估回滚成本。
6. 大事务回滚可能比执行更慢，不要重复提交新批处理。

7. 事故后必须补充索引、拆批、限流或改造事务边界。

26.42 开发阶段防锁表检查清单

提交任何写 SQL 前必须确认：

检查项	要求
<code>where</code> 是否命中索引	必须
是否可能全表扫描	禁止
是否按主键或唯一键更新	优先
是否大范围更新	必须拆批
是否大范围删除	必须拆批或归档
是否使用 <code>for update</code>	必须评审锁范围
是否低区分度字段更新	必须组合高区分度条件
是否有函数处理索引列	禁止
是否存在隐式类型转换	禁止
是否有 <code>or</code> 、 <code>like '%x'</code>	谨慎，通常禁止用于写 SQL
是否事务内远程调用	禁止
是否事务内长循环	禁止
是否固定加锁顺序	必须
是否检查影响行数	必须
是否设置事务超时	必须
是否有回滚和死锁重试策略	必须评估

26.43 MyBatis Plus 防锁表示例

推荐写法：

```
public void closeTimeoutOrders(Long tenantId, Long startId, Long endId,
    LocalDateTime timeoutTime) {
    txExecutor.requiredWithoutResult(() -> {
        int updated = orderMapper.update(
            null,
            new LambdaUpdateWrapper<OrderDO>()
                .eq(OrderDO::getTenantId, tenantId)
                .gt(OrderDO::getId, startId)
                .le(OrderDO::getId, endId)
                .eq(OrderDO::getStatus, OrderStatus.CREATED.name())
                .lt(OrderDO::getCreateTime, timeoutTime)
                .set(OrderDO::getStatus, OrderStatus.CLOSED.name())
                .set(OrderDO::getUpdateTime, LocalDateTime.now())
        );
    });
}
```



```
        if (updated > 1000) {  
            throw new BizException("单批更新数量超过限制");  
        }  
    });  
}
```

配套索引建议：

```
create index idx_order_tenant_status_id_time  
on t_order(tenant_id, status, id, create_time);
```

注意：

1. 使用租户、状态、主键范围控制扫描范围。
2. 单批更新数量受控。
3. 每批独立事务。
4. 任务外层负责推进 `startId` 和 `endId`。

27. 最终约定

1. 新增业务代码默认使用手动事务。
2. 新增业务代码不推荐使用 `@Transactional`。
3. 历史 `@Transactional` 代码允许逐步迁移，但修改时应优先改为手动事务。
4. 所有事务必须能从代码中直接看出边界。
5. 所有事务失败必须能确定是否回滚。
6. 所有状态更新必须检查影响行数。
7. 所有外部请求、消息消费、定时补偿必须具备幂等能力。
8. 所有事务相关代码必须经过回滚测试和并发风险审查。